

RELATÓRIO DE ESTUDO

Renan Homiak Guimarães
24000668-2

Aula 07: Expressões, Atribuição e Controle Paradigmas de Linguagens de Programação Engenharia de Software 02 de outubro de 2026

SUMÁRIO EXECUTIVO E INTRODUÇÃO AOS PARADIGMAS

Este documento apresenta uma leitura aprofundada, detalhada e estruturada dos principais conceitos trabalhados na Aula 07 da disciplina de Paradigmas de Linguagens de Programação. O foco central reside na investigação minuciosa de como diferentes linguagens tratam conceitos fundamentais da computação: a avaliação de expressões matemáticas e lógicas, as regras de atribuição de valores e o controle do fluxo de execução.

Em vez de encarar esses elementos meramente como açúcar sintático ou regras arbitrárias, a abordagem adotada enfatiza a teoria por trás das escolhas de projeto (*design choices*) de cada tecnologia. Compreender essas entrelinhas evita bugs críticos em sistemas corporativos e permite ao engenheiro de software antecipar comportamentos inesperados originados por precedências ocultas, coerções implícitas de tipos e desvios mal controlados.

1. VISÃO GERAL DO CONTEÚDO E MOTIVAÇÃO PRÁTICA

A Aula 07 concentrou-se em três pilares estruturais da programação imperativa e multi-paradigma:

1. **Expressões:** A combinação de operandos e operadores para a computação de novos valores.
2. **Atribuições:** Os mecanismos de vinculação de valores a variáveis e o uso de atribuições como expressões.
3. **Controle de Fluxo:** Estruturas sequenciais, de seleção, de iteração e desvios incondicionais.

O estudo baseou-se em uma abordagem exploratória de previsão e verificação. Antes de executar trechos de código em linguagens como Python, Java, JavaScript, C, PHP, Go e Rust,

os alunos foram desafiados a prever os resultados com base exclusivamente nas especificações formais de cada linguagem.

Casos Reais e o Impacto na Engenharia de Software

Pequenos deslizes na interpretação de expressões ou estruturas de controle frequentemente escalam para falhas catastróficas em sistemas de produção. Casos históricos ilustram perfeitamente essa vulnerabilidade:

- **O caso "goto fail" da Apple (2014):** Uma linha de código duplicada por engano (`goto fail;`) logo após a verificação de hash em rotinas de segurança do iOS e macOS fez com que o fluxo saltasse incondicionalmente a validação subsequente de assinatura digital. Como resultado, o sistema passou a aceitar certificados falsos como válidos, comprometendo a criptografia SSL/TLS de milhões de dispositivos.
- **O travamento dos tocadores Zune da Microsoft (2008):** No último dia de um ano bissexto, o algoritmo de cálculo de dias em um laço `while` entrou em um estado de avaliação em que nenhuma variável relevante mudava de valor. O laço perdeu a progressão e travou permanentemente os dispositivos em escala global.
- **O "Type Juggling" e comparações frouxas em PHP:** O uso do operador de igualdade ampla (`==`) em sistemas web legados permitia que hashes criptográficos iniciados com "0e" (interpretados internamente pela engine como notação científica para zero) fossem considerados equivalentes. Isso resultou em falhas graves de autenticação, permitindo que logins fossem burlados com senhas incorretas.

(Espaço reservado para inserção de captura de tela do slide: "Objetivo e ligação com o mundo real")

2. COMO UMA EXPRESSÃO É INTERPRETADA: PRECEDÊNCIA E ASSOCIATIVIDADE

A avaliação de expressões aritméticas e lógicas exige regras rígidas estabelecidas pelo projetista da linguagem para evitar ambiguidades. A ausência de regras universais faz com que uma mesma sequência de símbolos produza resultados completamente distintos dependendo da tecnologia empregada.

Precedência de Operadores

A precedência define qual operador deve ser avaliado primeiro quando diferentes operações concorrem em uma mesma expressão. Por exemplo, na expressão matemática clássica $a + b * c$, assumindo $a=3$, $b=4$ e $c=5$, a multiplicação possui prioridade sobre a adição. O

computador executa primeiramente $4 * 5 = 20$ e, em seguida, soma 3 , obtendo o resultado final 23 .

Diferentes abordagens históricas e paradigmáticas demonstram como isso varia:

- **Lisp:** Dispensa totalmente as regras tradicionais de precedência ao adotar uma notação prefixa estrita baseada em listas e parênteses explícitos $((+ a (* b c)))$, eliminando qualquer ambiguidade sintática.
- **APL:** Adota a regra de que todos os operadores possuem a mesma prioridade e a avaliação ocorre estritamente da direita para a esquerda. Assim, a expressão $3 * 4 + 5$ é interpretada como $3 * (4 + 5)$, resultando em 27 , contrariando a intuição matemática padrão.

Associatividade

Quando operadores de mesma prioridade aparecem consecutivamente, a associatividade determina a direção do agrupamento:

- **Associação à Esquerda:** Comum em operadores aritméticos básicos $(+, -, *, /)$, onde a expressão é avaliada da esquerda para a direita.
- **Associação à Direita:** Aplicada em operadores de exponenciação em várias linguagens $(**$ em Python, JavaScript e Ruby) e em operadores unários e de atribuição.

Um exemplo notável discutido na aula foi o cálculo da expressão $-2 ** 2$.

- Em **Python**, onde a exponenciação associa à direita, a expressão é interpretada como $-(2 ** 2)$, produzindo -4 .
- Em linguagens como **Visual Basic**, a associatividade e a precedência do menos unário podem diferir, enquanto o **Excel** interpreta $-^$ de maneira particular onde o menos unário vence a exponenciação, resultando em 4 para -2^2 .

(Espaço reservado para inserção de captura de tela do slide: "Precedência e associatividade na prática")

Diretriz de Engenharia: Na dúvida sobre precedências complexas entre diferentes linguagens, utilize sempre parênteses explícitos. Eles deixam a intenção do código cristalina para o compilador e para os demais desenvolvedores da equipe.

3. EFEITOS COLATERAIS E ORDEM DE AVALIAÇÃO DE OPERANDOS

Um conceito avançado abordado na aula diz respeito aos efeitos colaterais (*side effects*).

Avaliar uma expressão não se resume estritamente a computar valores estáticos; muitas vezes, envolve chamadas de funções que alteram o estado global do programa ou modificam variáveis externas.

O Problema da Ordem de Avaliação

Se uma expressão contém uma chamada de função que altera uma variável utilizada em outra parte da mesma expressão, o resultado final passa a depender diretamente da ordem em que os operandos são avaliados pelo compilador.

Considere o exemplo clássico adaptado do livro de Sebesta:

```
int a = 5;

int fun1(void) {

    a = 17; // Efeito colateral: altera a global 'a'

    return 3;

}

int main(void) {

    a = a + fun1(); // O resultado é 8 ou 20?

}
```

- **Em Java:** A especificação da linguagem garante estritamente que os operandos sejam avaliados da esquerda para a direita. O valor original de `a` (\$5\$) é lido antes da execução da função, resultando em $5 + 3 = 8$.
- **Em C (com o compilador moderno GCC 14):** O padrão da linguagem C *não define* uma ordem obrigatória para a avaliação dos operandos desta expressão. Consequentemente, o compilador pode optar por avaliar a função primeiro, modificando `a` para \$17\$, e só depois buscar o valor de `a`, resultando em $17 + 3 = 20$.
- **Em Go:** A especificação oficial também não garante a ordem de leitura das variáveis em relação à chamada de funções concorrentes ou dependentes, podendo gerar comportamentos não determinísticos.

Transparência Referencial

O antídoto conceitual para os efeitos colaterais é a **transparência referencial**. Uma função é referencialmente transparente se o seu resultado depende exclusivamente dos argumentos passados, sem alterar nenhum estado externo. Em ambientes puramente funcionais, $f(x) + f(x)$ pode ser otimizado com segurança para $2 * f(x)$, o que é impossível em linguagens imperativas repletas de efeitos colaterais ocultos.

(Espaço reservado para inserção de captura de tela do slide: "Ordem dos operandos e efeitos colaterais")

4. SOBRECARGA DE OPERADORES E SEUS RISCOS

A sobrecarga de operadores (*operator overloading*) permite que um único símbolo represente operações distintas dependendo dos tipos de dados envolvidos ou do contexto de execução.

O Operador de Adição (+)

O símbolo `+` é o exemplo mais difundido de sobrecarga nativa:

- Atua como adição matemática para tipos inteiros e de ponto flutuante.
- Atua como operador de concatenação de strings em linguagens como Java, JavaScript e Python.

Em **Java**, a mistura de tipos gera comportamentos sutis:

- `System.out.println('a' + 1);` resulta em `98`, pois o caractere `'a'` é promovido ao seu valor numérico na tabela Unicode/ASCII (`'a'` = 97) e somado aritmeticamente a `1`.
- `System.out.println("" + 'a' + 1);` resulta na string `"a1"`, pois a presença inicial de uma string vazia força a conversão das operações subsequentes para o contexto de concatenação.

Sobrecarga Definida pelo Usuário

Linguagens como C++, C#, Python, Rust e Kotlin permitem que o programador defina o comportamento de operadores para classes e estruturas personalizadas através de métodos especiais (como `__add__` e `__mul__` em Python).

- **Vantagem:** Torna o código matemático e científico extremamente expressivo e natural (ex: somar matrizes ou vetores usando `v1 + v2`).

- **Risco:** O abuso dessa facilidade pode mascarar operações de alto custo computacional ou criar semânticas contraintuitivas (como redefinir o operador `+` para realizar exclusões em banco de dados), prejudicando severamente a legibilidade.

(Espaço reservado para inserção de captura de tela do slide: "Operadores sobrecarregados")

5. CONVERSÕES DE TIPOS: ALARGAMENTO, ESTREITAMENTO E COERÇÃO

As conversões de tipos representam uma ponte delicada entre a intenção do programador e a representação binária executada pelo hardware.

Classificação das Conversões

1. **Alargamento (*Widening*):** Conversão de um tipo para outro com maior capacidade de representação (ex: de `int` para `float` ou `double`). Geralmente ocorre de forma implícita e segura, com pouca ou nenhuma perda de precisão.
2. **Estreitamento (*Narrowing*):** Conversão para um tipo de menor capacidade (ex: de `double` para `int` ou de `int` para `byte`). Envolve riscos inerentes de truncamento de valores ou estouro de capacidade (*overflow*).

Exemplos Práticos em Java e Outras Linguagens

- A conversão de um inteiro grande (`123456789`) para `float` e seu retorno posterior para `int` pode resultar em perdas sutis de precisão, gerando valores como `123456792` devido aos limites de bits na mantissa do padrão IEEE 754.
- Um cast explícito de `200` para o tipo `byte` em Java resulta em `-56` devido ao estouro no bit de sinal do complemento de dois.
- A divisão inteira (`1 / 2`) resulta em `0`, enquanto `1 / 2.0` resulta em `0.5`, demonstrando que o tipo dos operandos altera diretamente a natureza da instrução gerada.

Comparações entre Paradigmas de Tipagem

- **C e C++:** Realizam coerções automáticas agressivas em praticamente qualquer direção, muitas vezes emitindo apenas avisos em vez de erros de compilação.
- **Java e C#:** Permitem alargamentos implícitos controlados, mas exigem casts explícitos para estreitamentos.
- **Go, Rust, ML e F#:** Adotam uma postura estrita de segurança de tipos. Nenhuma coerção implícita é tolerada; somar um inteiro de 32 bits com um inteiro de 64 bits ou converter tipos requer chamadas de conversão explícita em código.

(Espaço reservado para inserção de captura de tela do slide: "Conversões de tipos: alargamento, estreitamento e coerção")

6. COERÇÃO AUTOMÁTICA EM JAVASCRIPT E O "TYPE JUGGLING" DO PHP

Linguagens dinâmicas priorizam a flexibilidade através de mecanismos complexos de conversão implícita em tempo de execução.

Armadilhas no JavaScript

O JavaScript é amplamente conhecido por suas regras bizarras de coerção implícita:

- `"5" - 2` resulta em `3` (força o operador de subtração a converter a string para número).
- `"5" + 2` resulta em `"52"` (força a conversão para string devido ao operador de adição sobrecarregado).
- `true + 1` resulta em `2` (booleano convertido para `1`).
- `0 == ""` avalia como `true`, e `0 == "0"` também avalia como `true`. No entanto, `"" == "0"` avalia como `false`, provando que a igualdade frouxa (`==`) **não é transitiva**.

Para contornar esses problemas, o padrão moderno do ecossistema JavaScript exige o uso rigoroso de operadores de igualdade estrita (`===` e `!==`), que proíbem qualquer coerção implícita de tipos.

O "Type Juggling" e Falhas Críticas no PHP

O PHP gerencia tipos dinamicamente através do mecanismo de *type juggling*. Em comparações utilizando `==`, strings que começam com números seguidos da letra 'e' (ex: `"0e12345"`) são interpretadas pelo interpretador como valores em notação científica exponencial (`0 * 10^12345 = 0`).

Isso gerou vulnerabilidades históricas graves em sistemas de login corporativos onde hashes de senhas MD5 gerados aleatoriamente coincidiam com o formato exponencial. A verificação `if ($hash_armazenado == $hash_digitado)` avaliava como verdadeira mesmo para senhas incorretas. A solução definitiva consiste na adoção mandatória de operadores estritos (`===`).

(Espaço reservado para inserção de captura de tela do slide: "Coerção implícita: JavaScript e PHP")

7. BOOLEANOS, AVALIAÇÃO DE CURTO-CIRCUITO E OPERADORES MODERNOS

A representação de valores lógicos de veracidade diverge profundamente entre as famílias de linguagens de programação.

O Conceito de Falsidade e Verdade (*Truthiness*)

- **C, C++ e JavaScript:** Não possuem um tipo booleano primitivo nativo em suas origens tradicionais (C utilizava inteiros onde `0` representava falso e qualquer valor diferente de zero representava verdadeiro). Em linguagens dinâmicas modernas, conceitos de *falsy values* incluem `0`, `0.0`, `""`, `[]`, `{}`, `null` e `undefined`.
- **Java, C#, Go e Rust:** Adotam tipagem booleana estrita. Expressões condicionais em estruturas de decisão aceitam exclusivamente valores estritamente booleanos (`true` ou `false`). Tentar compilar uma instrução como `if (n)` onde `n` é um número inteiro gera um erro imediato de compilação.

Avaliação em Curto-Circuito (*Short-Circuit Evaluation*)

Operadores lógicos como `&&` (E) e `||` (OU) realizam avaliação em curto-circuito: a execução é interrompida assim que o resultado lógico global pode ser determinado com base no primeiro operando.

- **Proteção de Ponteiros Nulos:** Em Java, a expressão `s != null && s.length() > 0` é segura. Se `s` for `null`, o primeiro termo avalia como falso e o segundo termo (`s.length()`) sequer é executado, evitando uma temida `NullPointerException`. Caso o programador utilize o operador bitwise não-curto-circuito (`&`), ambos os lados são avaliados e a exceção ocorre.
- **Operadores Modernos (JavaScript):** O operador de encadeamento opcional `?.` permite acessar propriedades profundas de objetos sem disparar erros caso um nó intermediário seja nulo. O operador de coalescência nula `??` fornece um valor padrão alternativo **apenas** quando o operando analisado for explicitamente `null` ou `undefined`, evitando que valores válidos como o número `0` ou strings vazias sejam descartados incorretamente (problema comum ao utilizar o operador lógico `||`).

(Espaço reservado para inserção de captura de tela do slide: "Avaliação em curto-circuito")

8. ATRIBUIÇÃO: MAIS DO QUE APENAS GUARDAR UM VALOR

A sentença de atribuição é o coração da computação imperativa, mas sua sintaxe esconde complexidades semânticas significativas.

Atribuição como Expressão

Em linguagens da família C (C, C++, C++ e JavaScript), a atribuição `=` é formalmente uma **expressão** que retorna o valor atribuído. Isso permite construções encadeadas e compactas como:

- `sum = count = 0;`
- `while ((ch = getchar()) != EOF) { ... }`

No entanto, essa característica abre brechas para erros clássicos de digitação. Escrever acidentalmente `if (x = y)` em vez de `if (x == y)` compila perfeitamente: o valor de `y` é atribuído a `x`, e o resultado da expressão é testado como condição, gerando comportamentos lógico-errôneos difíceis de depurar.

Abordagens Alternativas em Outras Linguagens

- **Java e C#:** Permitem que a atribuição seja uma expressão, mas restringem rigorosamente o tipo de retorno nas condições de estruturas de controle, impedindo o erro do `if (x = y)`.
- **Python, Go e Rust:** A atribuição **não é uma expressão**. Em Rust, por exemplo, uma atribuição avalia para o tipo unitário `()`, tornando a instrução `if x = 5` um erro de compilação intransigente.
- **Python (Operador Morsa `:=`):** Introduzido no Python 3.8, o operador de atribuição expressional (*walrus operator*) permite atribuir valores a variáveis dentro de uma expressão de forma controlada (ex: `if (n := len(dados)) > 5:`).

(Espaço reservado para inserção de captura de tela do slide: "Sentenças de atribuição")

9. ESTRUTURAS FUNDAMENTAIS DE CONTROLE E O TEOREMA DE BÖHM-JACOPINI

O controle de fluxo determina a ordem em que as instruções individuais de um programa são executadas pelo processador.

O Teorema de Böhm-Jacopini (1966)

Demonstrou matematicamente que qualquer algoritmo computacional pode ser escrito utilizando estritamente três estruturas fundamentais de controle:

1. **Sequência:** A execução linear de instruções uma após a outra.
2. **Seleção:** A escolha entre caminhos alternativos com base em condições lógicas.
3. **Iteração:** A repetição de um bloco de código enquanto uma condição permanecer verdadeira.

Se apenas três estruturas bastam para computar qualquer função computável, por que as linguagens modernas incluem tantas outras variações (como `for`, `switch`, `do-while`, `foreach`, `break`, `continue`)? A resposta reside puramente na **legibilidade e facilidade de escrita** para o programador humano.

10. SELEÇÃO MÚLTIPLA E OS RISCOS DO "ELSE PENDENTE"

As estruturas de seleção condicional apresentam armadilhas sintáticas clássicas na engenharia de software.

O Problema do "Else Pendente" (*Dangling Else*)

Em linguagens como C, C++, Java e C#, a indentação do código não possui significado semântico para o compilador. Quando estruturas `if-else` aninhadas são escritas sem chaves delimitadoras, surge a ambiguidade de a qual comando `if` um `else` isolado pertence. A regra gramatical padrão estabelece que **o `else` pertence sempre ao `if` mais próximo** que ainda não possua um `else` associado.

Linguagens modernas como Perl, Go e Rust eliminam essa classe de erros ao tornarem o uso de chaves obrigatório em blocos condicionais.

Evolução da Seleção Múltipla (Switch e Match)

- **O `switch` tradicional do C:** Apresenta o comportamento problemático do *fall-through* implícito. Se o programador esquecer de colocar um comando `break` ao final de um caso, a execução continua fluindo inintencionalmente para os casos seguintes.
- **Java 14+:** Modernizou o `switch` introduzindo expressões com setas (`->`), que eliminam o *fall-through* e tornam o código mais seguro.

- **Rust (`match`):** Introduziu um casamento de padrões exaustivo (*exhaustive matching*). O compilador obriga o desenvolvedor a tratar absolutamente todas as possibilidades lógicas possíveis (ou incluir um curinga `_`), recusando a compilação caso algum cenário fique descoberto.

(Espaço reservado para inserção de captura de tela do slide: "Seleção múltipla: do switch de C ao match")

11. LAÇOS DE REPETIÇÃO: CONTADORES, ITERADORES E A ARMADILHA DO LAÇO INFINITO

Os laços de repetição automatizam tarefas repetitivas, mas exigem rigor analítico quanto às suas condições de terminação.

Diferenças entre Paradigmas de Laço

- **Laço `for` em C:** Funciona essencialmente como um laço lógico disfarçado, onde a variável de controle pode ser manipulada de forma arbitrária tanto no cabeçalho quanto no corpo interno da rotina, gerando saltos inesperados na contagem.
- **Laço `for` em Python:** Atua estritamente como um iterador sobre sequências (*iterables*). Modificar a variável de controle dentro do corpo do laço (`range`) não altera a progressão da sequência percorrida.
- **JavaScript (`for...in` vs `for...of`):** Confundir essas duas construções ao iterar sobre arrays gera bugs silenciosos graves. O `for...in` percorre as *chaves/propriedades* enumeráveis de um objeto (retornando índices em formato de string), enquanto o `for...of` percorre diretamente os *valores* armazenados nos elementos do array.

A Lição Histórica do Zune

Conforme citado na introdução, laços lógicos sem progresso garantido levam ao travamento de sistemas inteiros. A análise do bug do Zune reforça o checklist indispensável ao projetar um laço:

1. Qual é a condição exata de entrada?
2. Qual variável controla o progresso da repetição?
3. Essa variável sofre alteração garantida em todas as iterações?
4. Existe um cenário lógico onde a condição de saída nunca é alcançada?

(Espaço reservado para inserção de captura de tela do slide: "Laços controlados por contador")

12. DESVIOS INCONDICIONAIS (GOTO) E COMANDOS PROTEGIDOS

O comando de desvio incondicional `goto` foi o centro de um dos debates mais célebres da história da ciência da computação, impulsionado pelo artigo de Edsger Dijkstra em 1968, "*Go To Statement Considered Harmful*".

O Perigo do GOTO

O uso indiscriminado de saltos incondicionais corrompe a estrutura lógica do software, transformando o fluxo de execução em uma teia caótica de complexidade ciclomática inadministrável (*spaghetti code*). O caso crítico da Apple em 2014 ("goto fail") evidenciou que mesmo engenheiros experientes podem falhar ao auditar códigos dependentes de múltiplos saltos de desvio.

Alternativas Modernas

Linguagens modernas substituíram o `goto` por desvios estruturados e restritos, como `break`, `continue`, `return` e mecanismos avançados de tratamento de exceções. Go mantém um `goto` altamente restrito (que não permite saltar sobre declarações de variáveis ou para fora de escopos de funções), enquanto Java aboliu completamente o comando.

Comandos Protegidos de Dijkstra

Como alternativa matemática aos desvios caóticos, Dijkstra propôs os comandos protegidos (*guarded commands*), baseados em premissas de execução não determinística onde múltiplas condições (*guardas*) podem ser avaliadas simultaneamente. Essa base teórica inspirou arquiteturas de concorrência modernas, como o canal `select` da linguagem Go, que gerencia múltiplos canais de comunicação de forma concorrente e segura.

(Espaço reservado para inserção de captura de tela do slide: "Desvio incondicional: o goto")

13. ATIVIDADE PRÁTICA DE FIXAÇÃO: ANÁLISE DE BUGS E SÍNTESE FINAL

A aula prática foi encerrada com a análise de trechos de código desafiadores projetados para testar o conhecimento profundo das regras gramaticais das linguagens:

1. **JavaScript (for...in em arrays):** Demonstrou que iterar arrays com `for...in` concatena índices incorretamente em vez de somar valores numéricos.
2. **Python (Parâmetros padrão mutáveis e curto-circuito):** Evidenciou armadilhas ao utilizar expressões como `d = d or 10`, onde passar o valor numérico `0` (válido para desconto nulo) era interpretado incorretamente como falso, aplicando o padrão de `10`.
3. **Java (switch sem break):** Mostrou o impacto do *fall-through* em atribuições de strings.
4. **C (Ponto e vírgula indevido):** Ilustrou o perigo de colocar um ponto e vírgula logo após a condição de um `if (saque > saldo);`, fazendo com que o bloco subsequente fosse executado incondicionalmente.
5. **Go (Sombreamento de variáveis):** Destacou os riscos do operador de declaração curta `:=` dentro de escopos aninhados.
6. **Java (Divisão inteira mista):** Demonstrou o truncamento em cálculos de taxas percentuais utilizando inteiros.

(Espaço reservado para inserção de captura de tela do slide: "Exercício: encontre o bug (1/3)")

(Espaço reservado para inserção de captura de tela do slide: "Exercício: encontre o bug (2/3)")

(Espaço reservado para inserção de captura de tela do slide: "Exercício: encontre o bug (3/3)")

Tabela-Resumo: Síntese de Armadilhas e Soluções nas Linguagens

Tema Abordado	Armadilha Potencial	Mecanismo de Mitigação / Solução
Precedência	Interpretação incorreta de operadores (<code>-2**2</code>)	Uso obrigatório de parênteses explícitos
Efeitos Colaterais	Avaliação não determinística de funções (<code>a + fun()</code>)	Adoção de transparência referencial e funções puras
Conversões	Perda de precisão e coerções silenciosas em C/PHP	Tipagem estrita e conversões explícitas em Go/Rust
Booleanos	Falsidade implícita de valores vazios (<code>0, ""</code>)	Checagens estritas e operadores de coalescência (<code>??</code>)
Atribuição	Atribuição acidental em condições (<code>if (x = y)</code>)	Proibição de atribuição como expressão (Python/Rust)
Seleção	<i>Fall-through</i> no switch e <i>else pendente</i>	Uso de chaves obrigatórias e padrão <code>match</code> exaustivo

CONSIDERAÇÕES FINAIS E REFLEXÃO PROFISSIONAL

O estudo detalhado da Aula 07 consolida a percepção de que a programação vai muito além da escrita mecânica de sintaxe. Expressões, atribuições e estruturas de controle encapsulam decisões de projeto históricas que impactam diretamente a corretude, a segurança e a manutenibilidade de softwares corporativos.

Para o engenheiro de software moderno, cultivar o hábito de investigar as regras gramaticais e as especificações de projeto de cada linguagem representa a diferença entre construir sistemas frágeis, vulneráveis a falhas obscuras, e desenvolver soluções robustas, previsíveis e altamente confiáveis.

REFERÊNCIA BIBLIOGRÁFICA

- SEBESTA, Robert W. **Conceitos de Linguagens de Programação**. 11. ed. Porto Alegre: Bookman, 2018. Capítulos 7 e 8, p. 299-361.
- MATERIAL DIDÁTICO DA DISCIPLINA. **Aula 07: Expressões, Atribuição e Controle**. Engenharia de Software, 2026.